

Multi-Agent 协作设计规约 (v0.2)

配套 v0.0 spec、confidence ledger、LLM compile 规约。解决问题：从“单 agent 工具”升级成“多 agent 安全协作平台”——让多个 agent 能同时演化同一份知识、不打架、可追溯、能审计、能撤销。状态：v0.0 / v0.1 不实施；v0.2 进入 P0。前置依赖：v0.0 的 git-backed vault + patch 概念雏形、v0.1 的 confidence ledger（事件流是核心基础）、v0.1 的 LLM compile（patch 是协作的基本单位）、v0.0 spec § 0.6 投影层原则。

0. 这份文档要回答的本质问题

给 agent / Codex 读者：先理解这一节，再往下读。这份文档剩余所有设计都是这一节的展开。

0.1 为什么 git 不够

人很容易说“我已经有 git 了，多 agent 写同一个 repo 不就行了？”——这是个本质上的误解。**git** 解决的是“文件变更的历史可追溯性”，不解决“多个写入者的并发协调”。具体：

- git 检测冲突但不消解冲突——它把冲突原样扔给人手动 merge
- git 不知道谁有权写什么——任何能 push 的人都能改任何文件
- git 不验证写入者身份——commit author 字段是字符串，可以伪造
- git 没有速率限制——一个 runaway agent 一小时能 push 一万个 commit
- git 不支持部分撤销——发现某个 agent 一周内的判断系统性错误，只能 revert 整段历史包括别人的好工作

人类的 git 工作流靠社会层规范填这些洞——code review、PR 模板、PR-bot、CODEOWNERS、人会累会停。Agent 没有这些。**Agent 不会累、不会自我节制、不会“今天先到这”、不会主动停下来手动 rebase**。所以 multi-agent 系统不是给 git 加点封装，而是要补 git 故意没做的那一层：**并发协调 + 身份信任 + 速率控制 + 选择性撤销**。

0.2 设计的五个核心原则

读到任何具体设计感到困惑时，回到这五条对照：

原则 1：git 是协调协议，不是协调机制。所有跨机器的事实同步走 `git push / pull`，akb 不发明分布式同步。但 git 之上的“安全 apply”由 akb 自己管。

原则 2：所有协作动作都是 append-only 事件。patch、handoff、agent action、revoke——

全部是事件流，永不修改、永不删除。撤销靠追加 **revoke 事件**实现，不是物理删除。这是 v0.1 confidence ledger 哲学的延续：**事件历史是事实，状态是投影**。

原则 3：agent 身份是一等公民。 不是字符串字段，是有 registry、有公钥、有权限、有配额、有审计的实体。任何 agent 写入必须可追溯到具体的 agent ID 和签名。

原则 4：信任靠分级，不靠平等。 不同 agent 的写入权重不同、可见范围不同、verify 信号强度不同。“每个 agent 都平等”的设计在被一个故障 agent 污染后就崩溃。

原则 5：并发用 optimistic concurrency，不用锁。 Patch 显式带 base commit，apply 时校验。冲突时拒绝 + 重新 compile，不让 agent 等锁。这跟 git 的 push/rebase 模型同构，agent 不需要“懂分布式系统”也能正确使用。

0.3 五层设计概览

multi-agent 设计分五层，由低到高：

Layer 5: 共享知识库 (vault federation / branch)	← 跨团队 / 长期
Layer 4: 并发 read 语义 (freshness / scope / trace)	← agent 读
Layer 3: Agent 协作语义 (peer review / handoff / 路由)	← multi-agent 价值
Layer 2: Agent 身份 (registry / 签名 / 配额 / revoke)	← 信任
Layer 1: 并发写入 (patch base lock / 串行 apply)	← 不打架

Layer 1 + 2 是必须——没有这两层，多 agent 跑起来就是 race condition + 不可追溯写入的灾难。**Layer 3 是 multi-agent 真正的价值**——光不打架还不够，要能协作。**Layer 4 + 5 是优化和扩展**——按需引入。

下面五节按这个顺序展开。**Codex 读者注意：每节开头都有一个 “Mental Model” 子节，告诉你这一层的核心模型是什么，再读细节会顺畅很多。**

1. Layer 1：并发写入与冲突消解

1.1 Mental Model

类比：把 patch 当成 git 的 commit，apply 当成 git push。

- Git 的 push 用 commit hash 链做 fast-forward 检查——patch 的 apply 用 base commit +

page hash 做同样的检查

- Git 冲突时 push 被 reject, 让人 rebase——patch 冲突时 apply 被 reject, 自动触发 re-compile
- Git 用 optimistic concurrency——patch apply 同样

关键差异: **agent 不会手动 rebase**。所以 patch 的“等价 rebase”必须是系统自动做的: 直接用最新 vault 状态重跑这个 source 的 compile pipeline。这比代码 rebase 简单, 因为 source 还在、prompt hash 还在、是可复现的。

1.2 Patch 显式声明 base

v0.1 LLM compile 文档定义的 patch 文件 schema 扩展:

```
# .akb/patches/patch_042.yaml
id: patch_042
title: "...
status: proposed

# ===== Layer 1 新增 =====
base:
  commit: "a3f2c1d8e9b7..."          # 生成 patch 时的 vault HEAD
  pageHashes:                          # patch 改动涉及的每个 page 的 content_hash
    page_gc001: "sha256:b2c3..."
    page_gc002: null                   # null = patch 创建这页时它还不存在
  ledgerHeads:                         # 涉及 page 的 ledger jsonl 最后一行 hash
    page_gc001: "sha256:f4e5..."
# =====

source: ...
compileMeta: ...
changes: ...
lineage: ...
signature: ...                        # Layer 2 引入
```

为什么 page 级 hash 而不是只 commit hash: commit 级 stale 太敏感——其他 agent 改了不相干的 page 也会让所有 patch 失效。page 级允许“我们改的 page 别人没碰过, 安全”的精确判定。

为什么也记 ledgerHeads: page 的 markdown 可能没变, 但它的 confidence ledger 增加了 contradicted_by 事件——这会让 patch 里的决策 (基于“高 confidence”做的 supersede 判定) 变得不合理。ledger 头部 hash 是 patch 在 confidence 维度的 “base”。

1.3 apply 的四态决策

akb apply patch_042 的判定流程：

```
base.commit == HEAD?
|
├ YES → fast-forward apply
|
└ NO → 涉及的所有 page 的当前 hash == base.pageHashes?
      |
      ├── YES → safe rebase apply (等价于 git fast-forward over unrelat
      |
      └ NO → 涉及的所有 page 的当前 ledgerHeads 是否安全?
            |
            ├── YES (只多了 verified / decay_checkpoint) → soft a
            |
            └ NO (出现了 contradicted_by / superseded_by) → RECI
                  |
                  └ 自动触发 `akb compile --source <src> --replay
                      (用最新 vault state 重跑这个 source 的 compil
```

四态的意义：

- **fast-forward**：90% 场景，最快路径
- **safe rebase**：另一个 agent 改了不相干 page，我们的 patch 仍然有效
- **soft apply with re-review**：page 本体没变但 confidence 演化了，决策可能要重审，但不立刻 reject（避免 livelock）
- **reject + re-compile**：真冲突，把 patch 作废，让系统自己生成新 patch。**Agent 不需要做任何“重试”——re-compile 是自动的，agent 该干嘛干嘛**

1.4 Apply 全局串行锁

并发 compile 没问题（read-only），但 apply 必须串行：

```
.akb/locks/apply.lock
{
  "pid": 12345,
  "agent_id": "agent:storage-research-alvin",
  "patch_id": "patch_042",
  "acquired_at": "2026-05-14T10:30:00Z",
  "heartbeat_at": "2026-05-14T10:30:05Z"
}
```

实现要点：

- acquire 用 `O_EXCL` 创建文件，原子操作
- 持锁方每 5 秒更新 heartbeat
- 别的进程看到 heartbeat 老于 30 秒 → 锁失效，可以抢
- release 用文件删除
- 持锁 P95 < 500ms (apply 本身快)，不是性能瓶颈

为什么不用 **page-level** 锁：以 v0.0-v0.2 规模 (< 10k pages、< 10 agents 并发)，全局锁的性能开销不是瓶颈。page-level 锁的复杂度 (deadlock detection、锁顺序) 远超收益。**先全局串行，实测瓶颈再优化**。这条要写进设计纪律。

1.5 自动 re-compile 的细节

REJECT 路径里“自动 re-compile”不是简单的“重跑一遍”。要点：

- # 触发自动 re-compile 时记录的事件
- .akb/agents/<id>/audit.jsonl 追加 patch_rejected 事件
- 原 patch 移到 .akb/patches/rejected/patch_042.yaml
- 用新 base (当前 HEAD) + 同样的 source + 同样的 prompts + 同样的 model 重跑
- 新 patch 用新 ID (patch_043)，不复用旧 ID (保 audit 干净)
- 新 patch 的 yaml 里 ``supersedesPatch: patch_042`` 字段，串起来

`supersedesPatch` 链让人能 trace 一个 source 经过几次 compile 才成功 apply。如果超过 3 次 reject → 停止自动 re-compile，标记 `needs_human_intervention`，避免 livelock。

1.6 Multi-machine: git 是协调层

不要在 akb 内部做“分布式同步”。让 git 兜底：

```
# 在机器 A
akb compile --source src_007
akb apply patch_042
akb push                                # 包装 git push

# 在机器 B (之前)
akb pull                                # 包装 git pull + 投影 rebuild
akb compile --source src_008
akb apply patch_043
akb push                                # 如果 reject:
                                        # ↓
                                        # 1. git pull (自动)
                                        # 2. 投影 rebuild
                                        # 3. 检查 patch_043 是否还能 apply (走 § 1.3 四态决策)
```

```
# 4. 如果 safe → re-apply + push
# 5. 如果 reject → 自动 re-compile (走 $ 1.5) + apply +
# 整个过程对 agent 透明
```

关键点：multi-machine 场景里，git 是事实流的传输协议。akb 在它两端做“apply 安全性”和“投影一致性”。akb 不需要懂“分布式系统的 consensus”——consensus 就是 git remote 的 ref 状态。

2. Layer 2: Agent 身份与信任

2.1 Mental Model

类比：SSH 公钥 + sudoers + rate limit + audit log。

- SSH 公钥 = agent 公钥，用来验证写入身份
- sudoers = agent permissions，定义“这个 agent 能改哪些目录、能不能 supersede、能不能 verify”
- rate limit = agent 配额，防止 runaway
- audit log = agent action log，append-only，事后可追溯、可撤销

人类的“我相信我的同事”在 multi-agent 系统里要拆成可机器执行的具体规则。这一层就是把“信任”形式化。

2.2 Agent Registry

```
# .akb/agents/registry.yaml (入 git, 是事实源)
version: "1.0"
agents:
  - id: "agent:storage-research-alvin"
    type: "claude-code" # claude-code | codex | cursor | custom
    owner: "alvin@example.com"
    publicKey: "ssh-ed25519 AAAAC3..."
    createdAt: "2026-05-14T10:00:00Z"

  permissions:
    can_compile: true
    can_apply: true
    can_supersede: false # 不能建立 supersede 链 (这是危险操作)
    can_verify: false # 不能写 verified 事件
    can_revoke: false # 不能 revoke 其他 agent
    page_scope: # 只能写的目录
```

```

    - "pages/storage/**"
    - "pages/concepts/**"
excluded_page_scope:                # 显式禁止
    - "pages/_decisions/**"        # ADR 类必须人工

quotas:
    max_patches_per_hour: 20
    max_pages_modified_per_patch: 5
    max_llm_calls_per_day: 500
    daily_llm_budget_usd: 10.00

weights:                            # 这个 agent 产生事件的权重
    verification_boost: 0.02        # 写 verified 事件的 boost
    source_weight_modifier: 1.0     # 它 ingest 的 source 默认权重 × 此值
    contradiction_severity_cap: "minor" # 它最多只能标 minor contradiction

status: "active"                    # active | suspended | revoked

```

为什么入 git: registry 是事实，跟 markdown 同等级别。改 registry 需要 commit + 通常需要 review（用 GitHub 的 CODEOWNERS 卡 `.akb/agents/registry.yaml` 必须人 review）。这是 multi-agent 系统里**最敏感的文件**——拥有改 registry 权限就拥有改一切。

2.3 Patch 签名

patch 文件加签名段：

```

# patch_042.yaml 末尾
signature:
    agentId: "agent:storage-research-alvin"
    signedAt: "2026-05-14T10:30:00Z"
    algorithm: "ed25519"
    payloadHash: "sha256:abc123..." # 对 patch 内容（除 signature 段外）的 sha2
    signature: "base64:ZGVmZ2hp..."

```

实现：

- Agent 启动时从 `~/.akb/keys/<agent_id>.key` 读私钥（这个文件 600 权限，不入 git）
- `akb compile` 在产出 patch 时自动签名
- `akb apply` 在 apply 前必做的事：
 1. 解析 `signature.agentId`
 2. 查 registry 找该 agent 的公钥
 3. 校验签名匹配 patch 内容

- 4. 校验 agentId 的 status 是 active
- 5. 校验改动涉及的 page 都在 page_scope 里
- 6. 校验配额没超
- 7. 全部通过才 apply

任一项失败 → reject，写 apply_rejected 事件到 .akb/agents/<id>/audit.jsonl 。

为什么签名验签：防两件事——(a) agent A 冒充 agent B 写权威页面；(b) patch 在协作链路中（PR review、CI、handoff）被篡改。在团队场景必须；单人本地也要做，因为“agent 跑飞了改了一通”这种事，能确认每个 patch 都是 agent 自己签的，事后归因清晰。

2.4 Agent Audit Log

.akb/agents/<id>/audit.jsonl (入 git, append-only)

每条事件一行 JSONL：

```
{ "ts": "2026-05-14T10:30:05Z", "kind": "compile_started", "sourceId": "src_007", "model"
{ "ts": "2026-05-14T10:30:17Z", "kind": "compile_completed", "sourceId": "src_007", "pat
{ "ts": "2026-05-14T10:30:18Z", "kind": "patch_signed", "patchId": "patch_042" }
{ "ts": "2026-05-14T10:30:25Z", "kind": "apply_attempted", "patchId": "patch_042" }
{ "ts": "2026-05-14T10:30:25Z", "kind": "apply_completed", "patchId": "patch_042", "mode
{ "ts": "2026-05-14T11:15:00Z", "kind": "quota_check", "spent_today": 7.32, "budget": 10.
```

事件类型清单（必须穷举，让 Codex 知道写哪些）：

kind	何时写
compile_started / compile_completed / compile_failed	compile 三态
patch_signed	patch 签名完成
apply_attempted / apply_completed / apply_rejected	apply 三态
quota_check / quota_exceeded	配额相关
permission_denied	权限校验失败
revoked / restored	agent 状态变更

入 git 的好处：跨机器协作时，所有 agent 的行动历史都在 git 里。出问题时 git log

.akb/agents/ 就能复盘。

2.5 Agent Revoke 与影响撤销

发现某个 agent 系统性错误（prompt 出 bug、模型行为漂移、被攻击）后需要撤销：

```
akb agent revoke agent:storage-research-alvin --since "2026-05-01" \
  --reason "Prompt bug: contradict misjudged as supersede"
```

执行步骤（顺序很重要）：

1. **registry 标记**：该 agent status 改成 `revoked`，写 revoke 原因
2. **找影响范围**：扫描 `.akb/agents/<id>/audit.jsonl`，找出 `--since` 之后所有 `apply_completed` 的 patch
3. **找 ledger 事件**：扫所有 page 的 `.ledger.jsonl`，找 `actorId == agent_id` 且 `--since` 之后的事件
4. **追加 revoke 事件**：对每个事件追加一条 `revoked_event` 事件，引用原 event id
5. **重算 confidence_state**：所有受影响 page 的 confidence 投影重建（revoke 的事件在重算时被跳过）
6. **标记 patch 待重审**：受影响 patch 的 `applied` 状态保留（不退档），但加 `needsReReview: true` 字段，触发人或别的 agent 重审
7. **生成报告**：`.akb/agents/<id>/revoke-report-<timestamp>.md` 入 git

核心设计点：不删 markdown、不删 ledger 事件、不 revert git history。只追加 revoke 事件，让投影层在重算时跳过被 revoke 的事件。这是 append-only 原则的直接 payoff——能“撤销影响”而不丢历史。

revoke 是可逆的：

```
akb agent restore agent:xxx --since "..."
```

追加 `restore_event` 事件，重算投影。被 revoke 的事件重新生效。

2.6 配额检查的实现位置

配额检查必须在三个地方做（防止绕过）：

1. **compile 启动时**：检查每小时 patch 数 + LLM call 数预算
2. **apply 时**：再查一次（compile 完到 apply 之间可能跨小时）

3. 后台 **reconciler**: 每小时跑一次 `akb quota verify`, 扫所有 agent 的当日实际消耗, 对比 registry 限额, 超限的 agent 自动 `suspend`

`suspended` 状态: 配额超限触发, 可被 `akb agent restore` 恢复。区别于 `revoked`: `suspended` 的 audit 记录有效, `revoked` 的 audit 记录在投影里被跳过。

3. Layer 3: Agent 协作语义

3.1 Mental Model

到 Layer 2 为止, 多 agent 已经能不打架了。Layer 3 解决“光不打架还不够, 要互相帮助”——这才是 multi-agent 真正的价值。

类比: 现实工程团队的三种协作模式:

- **专长路由**: 不知道答案时找懂的人问 → § 3.2 specialty + delegate
- **互审**: 写完代码找同事 code review → § 3.3 peer review
- **任务交接**: 研究阶段做完交给实现阶段 → § 3.4 handoff ledger

这一层全部基于 patch、handoff、review 等结构化对象——agent 不能“非正式协作”, 所有协作都是可追溯的事件。

3.2 Specialty 声明 + delegate 路由

Agent 在 registry 里声明专长:

```
agents:
- id: "agent:storage-deep-research"
  specialties:
    - domain: "storage-systems"
      keywords: ["NAND", "FTL", "ZNS", "FDP", "GC", "wear-leveling"]
      confidence: 0.95
    - domain: "kv-cache"
      keywords: ["PagedAttention", "vLLM"]
      confidence: 0.80

- id: "agent:codebase-reverser"
  specialties:
    - domain: "code-analysis"
      keywords: ["call-graph", "AST", "design-doc-generation"]
      confidence: 0.90
```

新 MCP tool (仅暴露给有 `can_delegate` 权限的 agent) :

```
{
  "name": "delegate_to_specialist",
  "description": "When you encounter a question outside your specialty, delegate",
  "inputSchema": {
    "type": "object",
    "properties": {
      "task": { "type": "string", "description": "What you want done, in plain la",
      "topic_hint": { "type": "string", "description": "Keywords helping the rout",
      "max_wait_seconds": { "type": "integer", "default": 60 }
    },
    "required": ["task"]
  }
}
```

路由逻辑 (在 akb MCP server 里实现, 不在调用方 agent 里) :

1. 解析 `topic_hint`, 用 `specialties.keywords` 做匹配, 按 `specialty.confidence` 排序
2. 排除: 已 `suspended/revoked`、配额耗尽的 agent
3. 选 top-1 specialist
4. 写 `delegation_started` 事件到双方 audit log
5. 实际怎么“执行”取决于部署模式 (见 § 3.5)
6. specialist 完成后写 `delegation_completed` 事件
7. 返回结果给调用方, 含 `specialist_id` 和事件 id (让调用方能 cite)

关键: `delegate` 不让调用方 agent 选 specialist——选谁由 registry 决定。否则调用方 agent 可能基于幻觉选错。

3.3 Cross-agent Peer Review

人工 review 是 multi-agent 协作的最大瓶颈。让 agent 互审:

patch 状态机扩展:

```
proposed                                // agent A 刚产出
  ↓ peer_review_requested
peer_review_in_progress
  ↓ approved / changes_requested / rejected
peer_reviewed                           // agent B 出了 verdict
  ↓ (低风险 patch 自动)                ↓ (高风险 patch 需要)
```

```
applied                                human_approved
                                     ↓
                                     applied
```

patch yaml 增加 review 段:

```
peerReview:
  reviewerId: "agent:storage-cross-checker"
  reviewedAt: "2026-05-14T10:35:00Z"
  verdict: "approve"                    # approve | request_changes | reject
  confidence: 0.85                      # reviewer 对自己 verdict 的信心
  reviewedChanges:                      # 逐个 change 的判定
    - changeIndex: 0
      verdict: "approve"
      note: "Contradiction note is correctly inserted, original content preserved"
    - changeIndex: 1
      verdict: "request_changes"
      note: "Supersede target page_gc001 conflicts with active claims in page_gc0"
  generalNote: "Overall direction sound but change 1 needs cross-reference to pag
  signature: {...}                      // reviewer 也要签名
```

Reviewer 必须满足:

1. 不同于 **author**——agent A 写的 patch 不能 agent A 自己 review
2. 最好是不同模型 **family**——claude reviews codex、codex reviews claude；同模型自审容易共享 blind spot
3. 必须有 **specialty 重叠**——reviewer 至少在该 patch 的 topic 上有 specialty
4. `can_peer_review: true` 权限

低风险 patch（满足以下全部）→ peer_reviewed 后自动 applied:

- `classifyConfidence > 0.8`（来自 compile 阶段的 LLM 判定）
- 不涉及 supersede（contradict / extend / merge OK）
- 不修改 `_decisions/`、`_runbooks/` 等高敏感目录
- `reviewer.verdict == "approve"` 且 `reviewer.confidence > 0.7`

高风险 → 进入 `human_approved` 等待人审。

实现细节: peer review 不是同步操作。流程是:

1. compile 产 patch, 写 `peer_review_requested` 事件

2. 后台 reviewer dispatcher (独立进程或 cron) 扫描 `peer_review_requested` 状态的 patch
3. 按 specialty 匹配 reviewer
4. 调用 reviewer agent 执行 review (这一步异步、可能很慢)
5. reviewer 写出 review patch (小型 yaml)
6. dispatcher 把 review patch 合并到原 patch 文件, 更新状态

3.4 Handoff Ledger

这是 multi-agent 协作里最重要的设计——让 agent 之间能“接力”完成跨阶段的工作。

新数据模型 Handoff (入 git) :

```
# .akb/handoffs/handoff_017.yaml
id: handoff_017
fromAgent: "agent:storage-research"
toAgent: "agent:codebase-reverser"          # 可以是具体 id, 也可以是 specialty 选择器
createdAt: "2026-05-14T11:00:00Z"
status: "pending"                          # pending | claimed | in_progress | co

# 任务上下文
context:
  topic: "Adaptive GC threshold validation"
  question: "Verify the adaptive threshold proposed in patch_042 actually works i

# 相关知识链接
relatedPages:
  - pageId: "page_gc001"
    role: "current_design"
  - pageId: "page_gc002"
    role: "proposed_new_design"

# 已经做过的决策 (避免下一棒重新讨论)
priorDecisions:
  - "Adopted FAST 2026 adaptive threshold concept (patch_042 applied)"
  - "Decided NOT to remove fixed-threshold fallback path"

# 给下一棒的具体问题
openQuestions:
  - "Find all call sites of gc_should_trigger() in the codebase"
  - "Verify the threshold parameter is configurable per-namespace"

# 已有的 artifact
artifacts:
```

```

- kind: "patch"
  id: "patch_042"
- kind: "ledger_event"
  pageId: "page_gc001"
  eventId: "01JN..."

# 期望的产出
expectedDeliverables:
- kind: "page"
  suggestedPath: "pages/storage/gc-implementation-mapping.md"
  purpose: "Map design (page_gc002) to code locations"

signature: {...}

```

新 MCP tool:

```

{
  "name": "create_handoff",
  "description": "Hand off work to another specialist agent or to a future session",
},
{
  "name": "claim_handoff",
  "description": "Claim a pending handoff that matches your specialty. The handoff",
},
{
  "name": "list_pending_handoffs",
  "description": "List handoffs waiting to be claimed, optionally filtered by specialty",
},
{
  "name": "complete_handoff",
  "description": "Mark a handoff as completed, linking the deliverables (patches, pages, etc.)",
}

```

为什么这是 multi-agent 真正的护城河:

普通 agent memory 系统 (Mem0、SuperMemory 那些) 记的是“agent 自己的会话历史”。Handoff Ledger 记的是“工作的交接”——工作交给谁、为什么、当时的决策、接手要做什么、做完产生了什么。这正是工程团队协作的本质。

入 git 后, handoff 是人能直接 **review** 的协作记录。一个 agent 接手时不需要扫一遍历史 chat log 来“理解上下文”——上下文已经被前一棒结构化地交出来了。

Handoff 也要走 confidence ledger: completed handoff 给相关 page 写 `verified` 事件 (弱信号, 类比 CI 通过)。abandoned handoff 不写。

3.5 部署模式与“具体怎么执行”

§ 3.2 的 delegate 和 § 3.3 的 peer review 里反复出现“调用 specialist agent 执行”。**akb 不内置 agent 运行时**——它只管编排和审计，agent 怎么跑取决于部署模式：

部署模式	delegate / peer review 怎么发生
单机 + Claude Code/Codex	akb 写 handoff/review 到 .akb/handoffs/ 或 .akb/reviews/ ，下次任一 agent 调 list_pending_handoffs 时看到、claim、执行
单机 + 多 agent 守护进程	每个 agent 跑一个后台进程，watch .akb/dispatch/ 目录，看到自己 specialty 匹配的任务就接走
多机 + git 同步	handoff 入 git，push 后另一台机器 pull 到、那台机器的 agent claim 执行、push 结果
托管模式	akb 暴露 webhook，外部编排器（n8n/Temporal/自建）订阅事件、调度 agent

akb 自身只承担：编排、签名验证、配额、audit、状态机。**不承担：**agent 进程管理、模型 API 调用、调度策略。这是工程边界，明确写进 spec 让贡献者别越界。

4. Layer 4：并发 Read 路径

4.1 Mental Model

读不像写——没有锁竞争。但有两个问题：**(a) 投影层可能 stale**（apply 完成到投影重建之间的窗口）；**(b) 不同 agent 应该看到不同的视图**（基于信任、specialty、隐私）。

4.2 投影 freshness 透明化

接受最终一致性，但让调用方能看到 freshness：

MCP search_knowledge 返回结果加字段：

```
{
  "results": [...],
  "freshness": {
    "vault_head_commit": "a3f2c1d...",
    "projection_built_at": "2026-05-14T10:30:00Z",
    "lag_ms": 47,
    "consistency": "eventual"
  }
}
```

```
}
```

LLM 可以自己看到“我拿到的结果是 47ms 前的快照”。99% 场景 < 100ms，问题可忽略；偶尔 race 至少透明、可调试。

强一致选项（按需）：

```
{
  "name": "search_knowledge",
  "inputSchema": {
    "properties": {
      "consistency": { "enum": ["eventual", "strong"], "default": "eventual" }
    }
  }
}
```

strong 模式下，akb 阻塞到投影重建完成再返回。延迟可能到 100ms-1s。只在关键决策（如 ADR 类页面审议）用。

4.3 Scope 限定

```
{
  "name": "search_knowledge",
  "inputSchema": {
    "properties": {
      "scope": {
        "enum": ["all", "own_specialty", "trusted_only", "human_verified_only"],
        "default": "all"
      }
    }
  }
}
```

- all：默认，全部命中
- own_specialty：限定在调用 agent 的 specialty domain
- trusted_only：只返回来自高权重 agent / 人审议过的内容（confidence > 0.7）
- human_verified_only：只返回最近 90 天内有 human verify 事件的内容（用于高敏感决策）

让 agent 主动限制自己的信息暴露面——这跟人类的“我先去翻官方文档而不是 stackoverflow”是一个动作。

4.4 Read Trace (可选)

`.akb/agents/<id>/queries.jsonl` (默认关闭)

每次 query 记录 query 文本 + top-K hit page ids。开启需要：

```
akb agent trace agent:xxx --on
```

两用途：

- **调试**：发现 agent 给错答案，回溯当时它检索到什么
- **改进 ranker**：作为 `access_recency` 信号给 confidence-aware ranker 输入 (confidence ledger § 3.7)

默认关闭，原因：写盘成本 + agent 行为隐私。需要时显式打开。

5. Layer 5：共享知识库

5.1 Mental Model

类比：不是数据库分片，是 git submodule。跨团队共享的知识不是“放进同一个 schema 用 ACL 隔离”，是“独立 vault 之间通过 git 协议交换”。**akb 不做多租户**——一个 vault = 一个 git repo = 一个团队的事实空间。共享走 federation。

5.2 Vault Federation

```
# .akb/federations/upstream.yaml
version: "1.0"
upstreams:
  - name: "storage-team-shared"
    git: "git@internal:storage-shared/knowledge.git"
    sync: "pull-only" # pull-only | bidirectional
    mountAt: "pages/_shared/storage/"
    confidenceProxy: 0.9 # 上游来的内容在本地的初始权重
    refreshSchedule: "0 */6 * * *" # cron 格式，每 6 小时 pull 一次
```

操作：

```
akb federation sync # 手动触发所有 upstream pull
akb federation status # 看上次同步时间、最新 commit
akb federation diff storage-team-shared # 看上游有什么新内容
```

实现：

1. akb federation sync 把 upstream 的 git pull 到 `pages/_shared/<name>/`
2. 投影 rebuild 时把这部分也 index 进去
3. citation 自动带上 `vault: storage-team-shared` 字段，区分本地 / 上游来源
4. 本地 patch 不允许修改 `pages/_shared/` 下任何东西（除非 `federation.sync == bidirectional`）

bidirectional 模式（v0.3+，先不实现）：本地对 `_shared/` 的修改积累为 PR，定期 push 回上游。这是真正的“协作 vault”——但复杂度高，先做 pull-only 模式。

5.3 Branch Workflow

把 git 的 feature branch 语义用起来：

```
akb branch feature/adaptive-gc-research
# 切到分支，这个 agent 可以激进 compile/apply
# main 分支不受影响

akb merge feature/adaptive-gc-research --into main --require-review
# 走 PR 流程
# - 计算 diff（涉及的 page、ledger 事件）
# - 跑 eval：保证 retrieval 准确率不回归
# - 触发 peer review（如果配了）
# - 人或 reviewer agent 批准后 merge
```

实现上 akb branch 是 `git checkout -b` 的简单包装 + 投影切换（不同分支不同投影库，避免污染）。

5.4 Per-page ACL（可选，多数场景不用）

```
# page frontmatter
---
id: page_secret_x
visibility:
  read: ["agent:trusted-*", "human:*"]
  write: ["human:alvin@*"]
---
```

实现：MCP `search_knowledge` 按调用 `agent_id` 过滤命中结果。

多数场景不需要这个——团队内信任假设下，“我不阻止你读，但我记录你读过”（§ 4.4 Read

Trace) 比 ACL 更实用。ACL 容易膨胀成 SharePoint 风格的权限地狱。

6. 与 v0.1 confidence ledger / LLM compile 的关系

multi-agent 不是替代前两块，是在它们之上加一层。具体集成点：

6.1 Confidence Ledger 集成

- 所有 ledger 事件必须带 **actor_id**——v0.1 已经规约，v0.2 强化：actor_id 必须能在 registry 解析到，否则事件 reject
- **Agent 权重影响事件强度**—— verification_boost 、 source_weight_modifier 等 registry 字段在算 confidence 时被用上
- **Revoke 通过追加 revoked_event 实现**——不删事件，重算时跳过
- **contradicted_by 的 severity 受 agent 权限限制**——低权限 agent 最多写 minor，需要高权限 agent 或人写 major

6.2 LLM Compile 集成

- **Patch 必须签名**——v0.1 文档里 patch 没签名字段，v0.2 加上是 P0
- **Patch 必须带 base 信息**——v0.1 隐含 base = HEAD，v0.2 显式
- **Peer review 在 patch 状态机里**——proposed → peer_reviewed → applied 三态扩展
- **chunk lineage 加 agentId**——v0.1 已经定义了 derivedFrom，v0.2 新增字段
- **Compile delegate**——agent A 遇到自己不擅长的 source 可以 delegate 给 specialist compile，结果 patch 标 compiledBy 是 specialist

6.3 投影层影响

multi-agent 给投影层加了几张新表：

```
-- 这些都不入 git，从 registry.yaml / audit logs / handoffs/ 重建
CREATE TABLE agent_state (
    agent_id          TEXT PRIMARY KEY,
    status            TEXT NOT NULL,
    last_action_at    TEXT,
    -- 当日累计配额消耗
    patches_today     INTEGER NOT NULL,
    llm_calls_today   INTEGER NOT NULL,
    spent_usd_today   REAL NOT NULL
```

```

);

CREATE TABLE patch_state (
    patch_id      TEXT PRIMARY KEY,
    author_id     TEXT NOT NULL,
    reviewer_id   TEXT,
    status        TEXT NOT NULL,      -- proposed | peer_review_* | applied | rej
    base_commit   TEXT NOT NULL,
    risk_level    TEXT NOT NULL      -- low | medium | high
);

CREATE TABLE handoff_state (
    handoff_id    TEXT PRIMARY KEY,
    from_agent    TEXT NOT NULL,
    to_agent      TEXT NOT NULL,
    status        TEXT NOT NULL,
    claimed_at    TEXT,
    completed_at  TEXT
);

CREATE TABLE revocations (
    id            TEXT PRIMARY KEY,
    agent_id      TEXT NOT NULL,
    since_ts      TEXT NOT NULL,
    reason        TEXT NOT NULL,
    created_at    TEXT NOT NULL,
    restored_at   TEXT
);

```

跟 v0.1 已有投影表一样，都通过 `akb projection rebuild --all` 从 canonical 文件重建。

7. CLI 命令汇总（v0.2 新增）

命令	用途
<code>akb agent register <id></code>	注册新 agent（生成密钥对、写 registry）
<code>akb agent list</code>	列出所有 agent 状态
<code>akb agent revoke <id> --since <ts> --reason <text></code>	撤销 agent 自指定时间起的所有动作影响
<code>akb agent restore <id> --since <ts></code>	取消撤销

<code>akb agent suspend <id></code>	临时禁用，配额耗尽时自动触发
<code>akb agent trace <id> --on/--off</code>	打开/关闭 query trace
<code>akb apply <patch> [--force]</code>	apply patch，走 § 1.3 四态决策；-force 跳过 review gate
<code>akb push</code>	包装 git push，含安全 rebase 重试
<code>akb pull</code>	包装 git pull + 投影 rebuild
<code>akb handoff create / claim / list / complete</code>	handoff 操作
<code>akb review <patch></code>	当前 agent 对 patch 出 review
<code>akb branch <name></code>	创建工作分支
<code>akb merge <branch> --into <target> --require-review</code>	合并分支，走 review gate
<code>akb federation sync / status / diff</code>	上游 vault 同步
<code>akb quota verify</code>	后台 reconciler，校验所有 agent 实际配额

8. MCP 工具汇总（v0.2 新增）

v0.0 只有 2 个工具（`search_knowledge`、`get_page`），v0.1 加 patch 提议工具，v0.2 新增以下（严格控制总数 < 12，超过要拆 server）：

工具	用途
<code>delegate_to_specialist</code>	把任务转给 specialist agent
<code>list_pending_handoffs</code>	看有什么 handoff 可以接
<code>claim_handoff</code>	接一个 handoff
<code>complete_handoff</code>	完成 handoff，链接产出
<code>create_handoff</code>	创建新 handoff 给下一棒
<code>request_peer_review</code>	对自己的 patch 请求互审

MCP tool 描述里要明确写：所有写操作都会被签名、计入配额、写 **audit log**。Agent 不要尝试规避——所有协议层校验都在 **server** 侧，无法绕过。

9. 几个故意不做的设计

multi-agent 系统的反模式集中区，明确告诉自己不做：

- 1. 不做“投票共识”——多 agent 对同一问题独立给答案再 vote。这听起来民主，实际是浪费 LLM call + 把判断责任分散到没人负责。用 peer review 模型，author + reviewer 明确职责。
- 2. 不做“agent 之间的实时通信”——所有协作经 git / 文件系统中转。同步 RPC 让系统变成分布式 monolith，难调试难审计。
- 3. 不做“agent 自动学习别的 agent 的 prompt”——prompt drift 是质量的最大杀手。每个 agent 的 prompt 由人维护、入 git、过 review。
- 4. 不让 agent 改 registry——registry 入 git 且必须人 review。can_register: true 不在任何 agent 权限里。
- 5. 不做“agent 信誉自动调整”——基于历史表现自动调权重听起来 ML，实际是把“哪个 agent 更可信”这个判断委托给一个无人审议的算法。权重在 registry，人改。
- 6. 不做“统一 agent 运行时”——agent 进程怎么跑由部署模式决定 (§ 3.5)，akb 只管编排和审计。
- 7. 不让 agent 同时做 author 和 reviewer——这是 multi-agent 互审的核心约束，没有例外。
- 8. 不做“agent 跨 vault 操作”——一个 agent 一次只对一个 vault 工作。跨 vault 协作走 federation。

10. 实现工作量估算 (v0.2)

组件	估算
packages/agent-registry 新 package	1.5 天
ed25519 签名 / 验签	1 天
Patch base 字段 + 四态决策	2.5 天

Apply 全局锁 + heartbeat	1 天
Audit log 写入路径 + JSONL 投影	1.5 天
配额检查（三处 + reconciler）	2 天
akb agent register / list / revoke / restore / suspend / trace	2 天
Revoke 影响撤销算法（重算投影含跳过逻辑）	2 天
akb push / pull 包装 + 安全 rebase	1.5 天
自动 re-compile on REJECT	1.5 天
Specialty 路由 + delegate_to_specialist MCP tool	2 天
Peer review 状态机扩展 + reviewer dispatcher	3 天
request_peer_review / submit_peer_review MCP tools	1.5 天
Handoff 数据模型 + 4 个 MCP tools + CLI	3 天
MCP 返回值加 freshness + scope 参数	1 天
akb federation sync / status / diff	2 天
akb branch / merge 包装	1 天
Per-page ACL（基础版）	1.5 天
投影层新表 schema + rebuild 逻辑	1.5 天
文档 + 例子 + multi-agent eval set	2 天
合计	34 天 ≈ 7 周专注

是 v0.0 / v0.1 / v0.2 三块里最大的。但有几块可拆：

- **P0 子集**（不做 multi-agent 系统失败）：base lock + apply 串行 + registry + 签名 + audit + chunk lineage 加 agentId + 基础配额。约 12 天，2.5 周。
- **P1 子集**（multi-agent 真正出价值）：peer review + handoff ledger + delegate。约 9 天，2 周。
- **P2 子集**：federation + branch workflow + ACL + freshness/scope。约 7 天，1.5 周。

- **P3 子集**：trace、bidirectional federation。后续迭代。

实际推荐：P0 + P1 = 5 周里程碑发布。P2/P3 按真实需求拉。

11. 验证标准

实施 multi-agent 之后必须能演示：

1. **并发不打架**：两个 agent 同时跑 `akb compile` 不同 source，apply 都成功，git log 干净
2. **过时 patch 自动重试**：构造 race（A 改 `page_X`，B 基于旧 X 生成 patch），B apply 被 reject，自动 re-compile，新 patch 成功 apply
3. **签名验证**：手动篡改 patch 内容（不重签），apply 拒绝
4. **配额生效**：把 `max_patches_per_hour` 设为 2，让 agent 跑第 3 个 compile，apply 失败并记 `quota_exceeded`
5. **Revoke 影响撤销**：让某 agent 写 10 条 verified 事件造成 confidence 上升，revoke 后所有相关 page 的 confidence 回到 revoke 前水平（但 markdown 和 ledger 文件没动）
6. **Restore**：上一步 revoke 后 restore，confidence 又回到 revoke 前的高水平
7. **Peer review 双签**：patch 经过 author + reviewer 双签后 applied，patch 文件含两个签名段
8. **Author 不能自审**：让 agent A 尝试 review 自己的 patch，被拒绝
9. **Handoff 完整流程**：agent A 创建 handoff，agent B claim、做完、complete，整个流程入 git，handoff 状态机正确流转
10. **MCP scope 生效**：用 `scope: "human_verified_only"` 查询，结果全部是最近 90 天有 human verified 事件的 page
11. **Federation 只读**：从 upstream pull 知识，本地 patch 尝试改 `pages/_shared/` 下文件，被拒绝
12. **Multi-machine push reject 自愈**：两台机器各自 apply patch，第二台 push 被拒，自动 pull + rebase + re-apply + push 成功

任何一条做不到，说明 multi-agent 设计有漏洞，先修再继续。

12. 给 Codex / 实现 agent 的最后提示

如果你是被指派实现这份 spec 的 agent，下面这些是最容易写错的地方：

1. **配额检查必须在三处**（compile / apply / reconciler）。只在 compile 时查会被 race 绕过——agent 同时跑 10 个 compile，每个都查到“还有 20 个配额”，结果实际超 10 倍。
2. **签名校验在 apply 时做，不在 compile 时做**。compile 时签名，apply 时才校验。颠倒会让 patch 在 apply 前能被静默篡改。
3. **revoke 不删事件**，是追加 `revoked_event`。投影 rebuild 时跳过被 revoke 的事件。物理删除是 architectural violation。
4. **handoff 不是 RPC**，是文件落盘 + 状态机。不要用 socket / HTTP 实现 handoff——那样不能 git 同步、不能审计。
5. **apply 锁是文件锁，不是数据库锁**。SQLite 的写锁粒度对 multi-machine 没意义。`.akb/locks/apply.lock` 在所有节点 pull 后都能看到。
6. **peer_review_requested 的 patch 不能阻塞**——dispatcher 异步处理。同步等 peer review 会让单 agent 工作流也变慢。
7. **chunk lineage 加 agentId 是 P0**，跟 v0.1 spec 比这是一行 schema 的事。漏掉这一行会让后续所有审计断链。
8. **registry.yaml 入 git 是死规矩**。任何让 registry 在运行时被 agent 修改的设计都直接拒。

读完前 0 节（Mental Model）和这一节（实现陷阱），就可以从 Layer 1 开始一层层实现。每层完成后跑对应的验证标准（§ 11），过了再做下一层。

13. 一句话总结

multi-agent 不是给单 agent 系统加几个 feature，是直面 git 不解决的四个问题：并发协调、身份信任、速率控制、选择性撤销。

每一层都建立在前一层的不变量上：Layer 1 保证写不打架，Layer 2 保证写入者可信任、可撤销，Layer 3 让协作变成结构化对象（patch / review / handoff）而不是非正式 chat，Layer 4 让读操作透明可控，Layer 5 让 vault 之间能交换知识。

跟 git 的关系是：git 是事实流的协议（push/pull 同步、commit 历史是事实），akb 是协议之上的 application（签名、配额、审计、协作语义）。git 不解决的事，akb 解决；git 解决的事，akb 不重复发明。

这跟 v0.1 的“知识不腐烂 + 知识自己长在一起”是不同维度——v0.1 在时间维度和结构维度解决问题，v0.2 在主体维度解决问题。三块拼齐：**知识能积累、能演化、能多 agent 协作演化**——这才是“AI-Native Knowledge Compiler”在 multi-agent 时代成立的完整形态。

